

How to Trust Your Benchmark Results Again

2026-04-20

Datacenters - the windowless buildings behind every online purchase, post, and prompt - are consuming electricity at alarming rates. Critics see datacenters as environmentally disruptive, and recent construction projects have been delayed in part due to these concerns. While taking **CS8803: Datacenter Networks and Systems** this spring, we asked the question: could software make these facilities more energy efficient?

For our final project, we explored clever *load balancing algorithms* that distribute work amongst hundreds of servers. Because sinking a datacenter in the ocean or launching one into space was out of scope for the course, software optimizations felt like a more practical approach.

My job was to simulate thousands of users concurrently searching for hotels online while taking detailed measurements of servers' **power consumption** and response **latency** (think: time it takes to receive a confirmation message after clicking “book” on AirBnB).

What I didn't know going into this project is that real-world applications don't always behave the way we expect them to. Conducting *reproducible* experiments with these systems is tricky but nonetheless crucial to building trust in our solutions.

I believe the tips in this post are helpful not only to researchers, but for engineers in industry too! Just as researchers make claims in papers that must be backed by reproducible results, companies make guarantees about how their services will perform in the real world through *service-level objectives (SLOs)*.

How It All Started

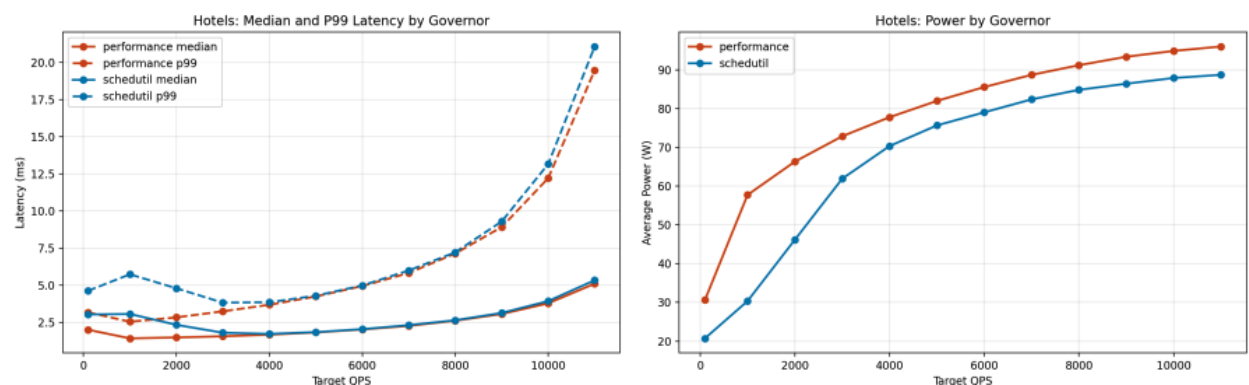


Figure 1: big if true: the one figure in our 6-page report that caught our professor's eye

The figure you're seeing above is what led me down a rabbit-hole of trying (and failing) to reproduce its findings. Simply put, the blue and orange lines in this plot represent different algorithms called *frequency*

governors. These frequency governors act similar to a speed limiter in a car: they intentionally limit the CPU clock rate (or top speed) to optimize for power utilization (or fuel economy/safety). In the plot on the right, we can see the `schedutil` governor (blue line) uses consistently less power than the `performance` governor (orange line). Meanwhile, the left plot shows that `schedutil` matches `performance` in terms of latency (i.e. how fast users can book hotels) at high loads.

Admittedly, I didn't know enough about frequency governors at the time to flag this discovery as interesting. The implication of these results is that we can gain **real power savings** for certain workloads by limiting server's clock rates and running them close to maximum load. Crucially, we don't have to sacrifice precious *p99 latency* (the dashed lines in the left plot), which is a metric datacenter operators tend to care about most.

It's no surprise that our professor encouraged us to reproduce this result in other real-world settings, because it might bring us one step closer to sustainable datacenters.

How to Conduct Reproducible Experiments

The tips I've compiled in this guide are based on my experience running experiments against a gRPC-based hotel reservation service, part of the larger DeathStarBench cloud microservices benchmark. Feel free to fork this repo if you'd like to follow along in the code.

I am trying to measure the median and p99 latency of the hotel reservation application while increasing the number of requests per second (RPS). The experiment finishes once the server has reached a point of *saturation*, which is the point at which all of its CPU resources are fully utilized.

1. Establish Baselines

As we add more variables to our experiments, baselines come in handy, giving us a way to sanity check our results. Baselines should be obtained from the exact same setup we plan to run real experiments against, as not every machine in a datacenter may have the same resources (which makes the datacenter environment *heterogeneous*). For example, machines may use CPUs from different vendors (AMD or Intel), have a different number of cores, or different bandwidth on their network interface cards (NICs). For my experiments, I am using two identical machines with the specs below (more on why this is important soon):

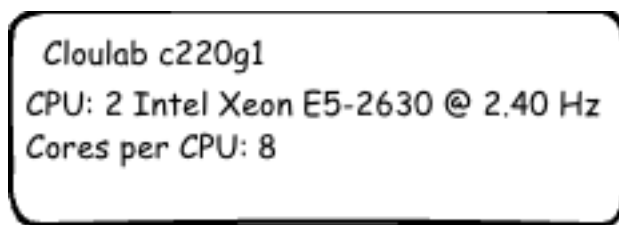


Figure 2: Specs of my testbed

In general, a baseline can be a simplified version of the algorithm you are experimenting with, or an algorithm that has a well-maintained open source implementation. Since I'm comparing algorithms that limit a CPU's clock rate, a natural baseline is an algorithm that lets the CPU use its maximum clock rate without any imposed limits. This algorithm is referred to as the `performance` governor in this post. To obtain my baseline measurements, I cloned the DeathStarBench repo and followed the instructions in the `README.md` for deploying the app inside docker containers.

After deploying the app, I had access to an HTTP server that I could send requests to, which would then make remote procedure calls to relevant microservices and return search results to the client. To send

thousands of these requests per second, I am using the `wrk2` HTTP load testing tool. For those following along, execute the `./scripts/install.sh` script in `hotelReservation` to install `wrk2`.

This is where the specs of the machines you use for testing can make or break your results: if the client has much fewer compute resources than the server (e.g. fewer threads), then it might not have the capacity to push the server to its true limits. As a result, the latency measures we obtain could be unreasonably high (e.g. on the order of seconds)

To start, I simulated 128 users collectively making 1,000 requests/second using `wrk2`:

```
../wrk2/wrk -D exp -t 4 -c 128 -d 50 -L -s ./wrk2/scripts/hotel-reservation/single-endpoint.lua 10.10.1
```

A minute later, I got back some useful latency measurements:

```
Test Results @ http://10.10.1.2:5000
  Thread Stats   Avg      Stdev     99%   +/- Stdev
    Latency    7.08ms    5.47ms   21.73ms   79.14%
    Req/Sec   254.77    95.55   500.00    67.37%
  Latency Distribution (HdrHistogram - Recorded Latency)
 50.000%    6.79ms <- median latency
 75.000%   10.83ms
 90.000%   14.59ms
 99.000%   21.73ms <- p99 latency
 99.900%   29.41ms
 99.990%   38.40ms
 99.999%   42.08ms
100.000%   42.91ms
```

There's our p99 latency in the fifth row from the bottom! It looks like 99% percent of requests completed in under ~22 milliseconds. These results are already super helpful because they give us an upper limit on the latency figures we should get. Anything much larger than these numbers can suggest that something might be wrong with our experimental setup.

2. Start out by Measuring a Single Path in Code

With our baseline established, we can get into the *real experiments*. There is just one catch: our hotel reservation application consists of multiple microservices that exchange messages to perform a search query. At companies like Netflix, applications consist of 700+ microservices, which can make troubleshooting anomalous results difficult.

To illustrate my point clearly, we can use an observability tool called Jaeger to visualize the code paths that are exercised for a search request. Jaeger allows us to trace the path a request takes through code in a way that print statements can't; using Jaeger, we can trace applications where a request may be passed through multiple containers (as is the case here) or multiple virtual machines scattered across a datacenter.

First, let's open the Jaeger dashboard in our browser (for those following along, run `./scripts/start_services.sh` and navigate to `http://localhost:5051`)

We can see that search requests complete within 2 milliseconds on average. See those colored squares under each trace? They represent the different microservices (geo, profile, rate, reservation) that our search requests depend on.

Next, let's click on one of the traces to get a better understanding of how a request passes through the code.

The visualization above is called a *dependency graph*, where circles represent microservices and arrows capture data dependencies. From this graph, we get a sense that searching for a hotel is not as straightforward as

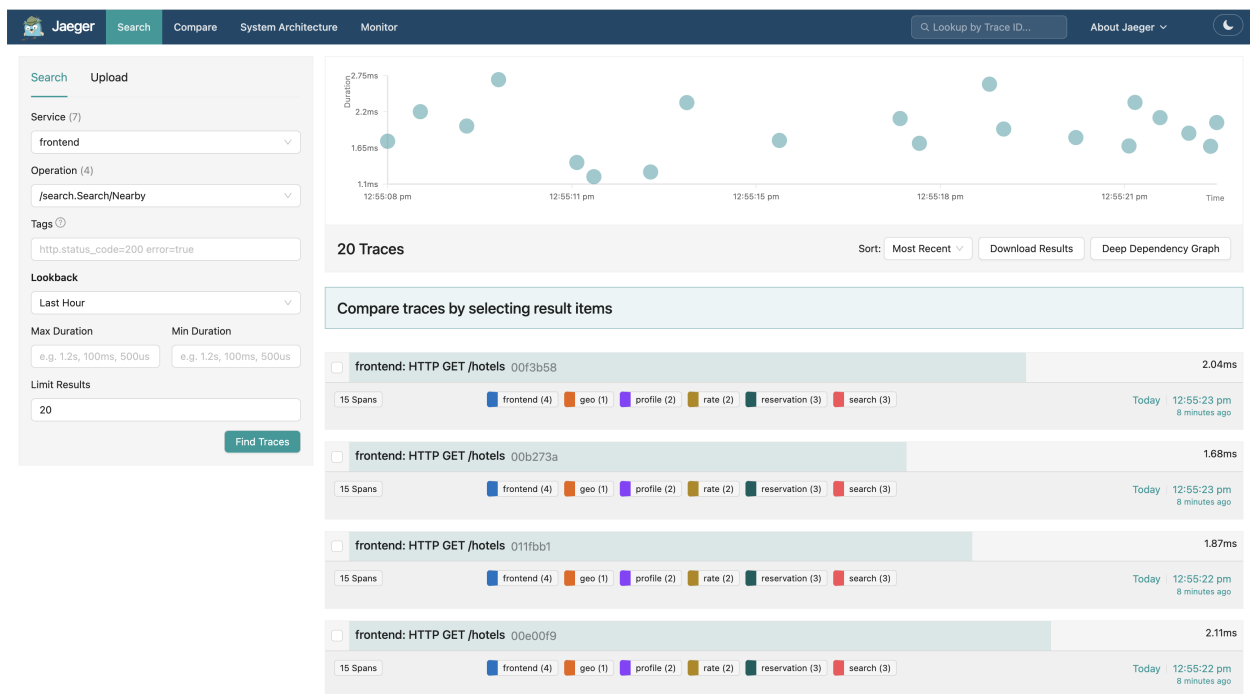


Figure 3: The jaeger dashboard

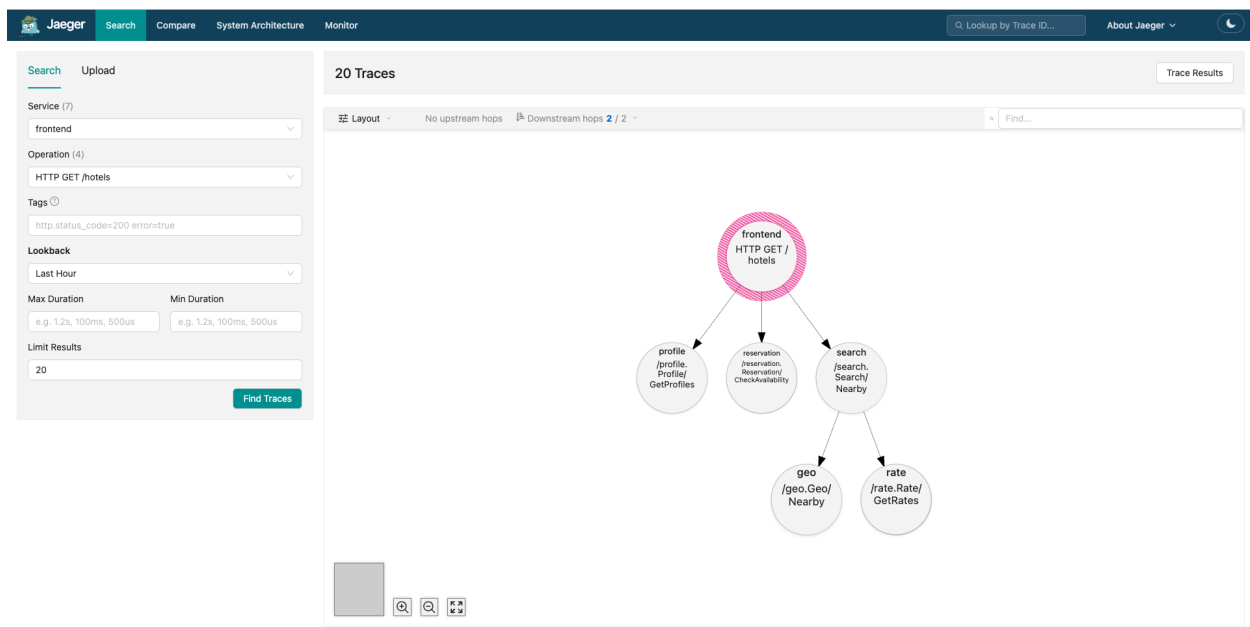


Figure 4: A dependency graph in Jaeger

previously thought; when a user wishes to search for a hotel, our application actually has to call three separate microservices to determine hotels that are:

1. Close by to the user's location
2. Within the user's price range
3. Available to book during the user's vacation

What you also don't see here is that each of the leaf microservices (i.e. the circles without arrows going out) are typically fetching data from a database (e.g. MongoDB) or a cache (e.g. Memcache). That's already a lot of moving pieces for a relatively straightforward search query! Now let's take a look at the code to make it simpler to debug odd results:

```
func (s *Server) searchHandler(w http.ResponseWriter, r *http.Request) {

    log.Trace().Msg("starts searchHandler querying downstream")

    searchResp, err := s.searchClient.Nearby(ctx, &search.NearbyRequest{
        Lat:      lat,
        Lon:      lon,
        InDate:   inDate,
        OutDate:  outDate,
    })

    reservationResp, err := s.reservationClient.CheckAvailability(ctx, &reservation.Request{
        CustomerName: "",
        HotelId:      searchResp.HotelIds,
        InDate:       inDate,
        OutDate:      outDate,
        RoomNumber:  1,
    })

    profileResp, err := s.profileClient.GetProfiles(ctx, &profile.Request{
        HotelIds: reservationResp.HotelId,
        Locale:   locale,
    })

    json.NewEncoder(w).Encode(geo.JSONResponse(profileResp.Hotels))
}
```

In the `searchHandler` function, we can see three calls to the `rate`, `reservation` and `profile` microservices. For my debugging experiments, I temporarily removed the calls to `reservation` and `profile`, modifying the search handler to only return hotel pricing data.

This modification did not seem to help however, as I could still see variance in the latency metrics between consecutive experiments. That's when I decided to go a step further and remove caching entirely from the critical path of a search request.

Making this simplification of the codebase is what revealed the flaw in my experimental setup: I was not restarting the `memcache` server in between trials, which gave the `schedutil` governor an unfair advantage because its cache had been warmed by prior `performance` trials.

For those unfamiliar with caches, they essentially “remember” data that an application tends to access frequently to make data accesses faster. While this sounds great, a somewhat annoying side-effect of caches for reproducibility is that they start “cold” and gradually get “warmer” over time as an application’s data access patterns stabilize.

That’s one part of the puzzle solved: `schedutil` matches `powerstat` in latency measurements despite using less power because it had a warmer cache. The second part of the puzzle is figuring out why `schedutil` uses less power than `powerstat` even at high load. Read on to find out!

3. Document Your Configs

When your config lives in terminal commands that hide in long slack threads, it becomes *much* harder to track the experiments you run. You might remember how our physics teachers in high school would be so picky about our lab notetaking:

Closer to the end of the project, I saved all of my experiment parameters in a python file and tracked changes to version control:

```
#!/usr/bin/env python3
import os
from pathlib import Path

# Default workload assignment and governor order.
DEFAULT_TARGETS = ["hotels", "recommendations", "reservation", "user"]
DEFAULT_GOVERNORS = ["performance", "schedutil"]

# SSH configuration for connecting to experiment nodes.
SSH_USER = os.environ.get("HOTEL_REMOTE_SSH_USER", "")
SSH_KEY_PATH = os.path.expanduser(os.environ.get("HOTEL_REMOTE_SSH_KEY", ""))

# Repository and artifact layout on the remote hosts.
REPO_ROOT = Path(__file__).resolve().parents[2]
REMOTE_REPO_ROOT = os.environ.get("HOTEL_REMOTE_REPO_ROOT", str(REPO_ROOT))
REMOTE_SCRIPT = "hotelReservation/scripts/run_power_sweep.sh"

# Experiment defaults.
HOST_URL = os.environ.get("HOTEL_REMOTE_HOST_URL_TEMPLATE", "http://%h:5000")
THREADS = 4
CONNECTIONS = 128
RATES_SPEC = "1000:20000:1000"
WRK2_DURATION=30
POWERSTAT_INTERVAL = 0.5
POWERSTAT_SOURCE = "auto"
SETTLE_SECONDS = 5

# Result locations.
RESULTS_ROOT = REPO_ROOT / "results" / "distributed_power_sweeps"
LOCAL_OUTPUT_DIR = str(RESULTS_ROOT)
REMOTE_OUTPUT_BASE = os.environ.get("HOTEL_REMOTE_OUTPUT_BASE", "")
```

Tracking changes to my configuration helped me go back in the revision history and discover a timing bug in my experiment. In the file above, you can see that the duration of my load test (in `WRK2_DURATION`) is

set to 30 seconds. However, the `powerstat` tool I'm using can only capture measurements over 60-second intervals at minimum, which is not immediately clear from the config.

This discrepancy is what leads to `powerstat` collecting some power measurements while the server is idle, and because the delay in collecting measurements was not uniform across all trials, there were times where `schedutil` had more idle measurements than `powerstat`, giving the illusion of `schedutil` using less power on average than `powerstat`. That

Version control is your friend! I found myself branching off versions of code to get back to states of the codebase .

Conclusion

By establishing baselines, starting small, and documenting my experiments, I was able to pinpoint the flaws in my setup, and fix my scripts to obtain results that myself and my colleagues could reproduce. (see below):

